

git简易教程

git——分布式版本管理工具，分布式就是各自计算机上有相应的版本各自进行修改和操作，最后再进行版本确定和合并

还有一种是集中式版本管理：这种一般是从云端服务器各自拷贝一份版本文件，进行修改后再上传到云端服务器，这种极容易出现因为网络问题而无法进行工作的问题

注意：分清远程仓库和本地仓库

- 远程仓库：一般是 github/gittee 等大型开源项目管理仓库
- 本地仓库：就是本地 git 版本管理的仓库

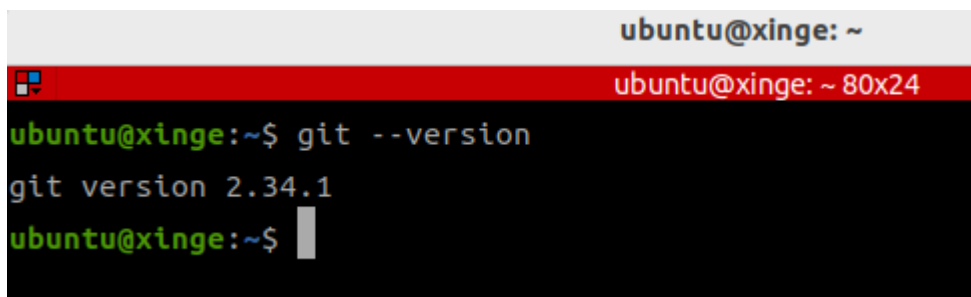
一. 初始化配置

在ubuntu22.04系统下，通过以下命令：

```
sudo apt install git
```

安装完毕之后可以用下面命令来查看git的版本：

```
git --version
```

A terminal window screenshot with a black background and green text. The prompt is 'ubuntu@xing: ~'. The command 'git --version' has been entered and executed, resulting in the output 'git version 2.34.1'. The cursor is at the end of the prompt line.

```
ubuntu@xing: ~  
ubuntu@xing: ~ 80x24  
ubuntu@xing:~$ git --version  
git version 2.34.1  
ubuntu@xing:~$
```

git安装完毕之后，就需要进行相关的初始化配置：

```
git config --global user.name <your name>
```

```
git config --global user.email <your email>
```

```
git config --global credential.helper store
```

```
git config --global --list
```

第一个第二个命令都是用于初始化git相关配置

- --global：全局配置，对所有仓库都生效
- --system：系统配置，对该计算机上所有用户和所有仓库都生效（很少去用）
- 如果省略：本地配置，只对当前仓库生效

一般使用的是 `--global`

第三个命令用于对name和email进行保存，无需下次再次进行 `config` 输入

第四个命令可以查看 `config` 的结果

```
ubuntu@xinge:~$ git config --global --list
user.email=2237200336@qq.com
user.name=xinge
safe.directory=*
credential.helper=store
```

二. 新建仓库

先来简单介绍一下仓库相关的概念，仓库——repo，仓库就是一个目录，但是里面所有的文件都可以被git进行跟踪，以便于进行版本相关管理

1. 方式1:

```
git init
```

用于在本地进行仓库初始化，这时候在你目录下就会有一个 `.git` 这个隐藏目录，这个隐藏目录，可以用以下命令进行查看：

```
ls -a
```

同时，我们还可以用

```
git init <repo_name>
```

直接创建一个带有 `.git` 隐藏目录的 `repo_name` 目录

2. 方式2

```
git clone
```

这个是从远程服务器上直接克隆下别人的仓库进行操作，这个是后续接触最多的创建仓库的方式

三. 工作区域和文件状态

1. 工作区域

一般分成3个区域：工作区，暂存区，本地仓库

- 工作区：就是本地计算机上进行文件操作的区域
- 暂存区：就是中间区域，用于存放修改过后准备提交的文件区域
- 本地目录：就是git进行管理的相关区域，主要进行版本控制

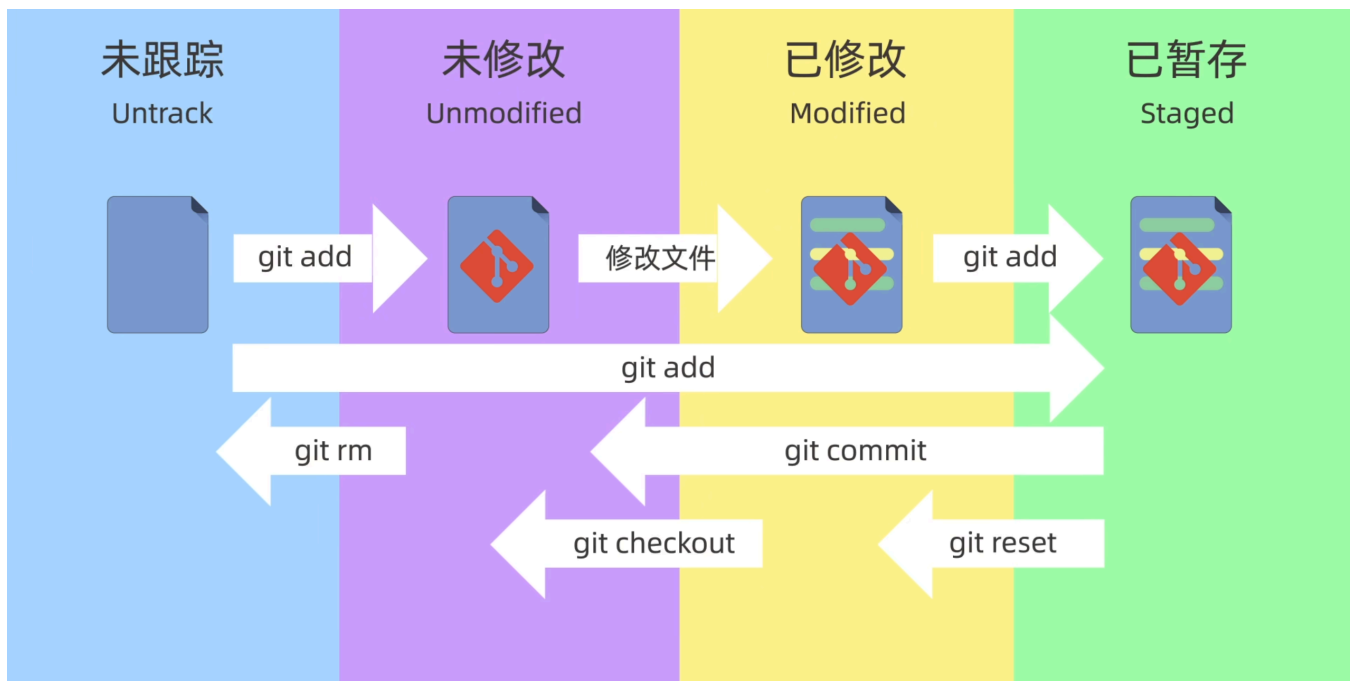
一般工作区通过 `git add` 将工作区进行操作后的文件上传到暂存区，然后利用 `git commit` 命令将暂存区的已经修改过的文件提交到本地目录进行git版本控制

2. 文件状态

文件状态分为四个状态：

- 未跟踪：就是在工作区下没有进行git版本控制的相关文件
- 未修改：已经提交到git上进行版本控制的相关文件但是没有进行修改的文件
- 已修改：已经进行修改且git版本控制的相关文件
- 已暂存：上传到暂存区的且git版本管理的相关文件

下面这张图很好地表示了各个文件的状态：



四. 添加和提交文件

承接三而来，这里详细说明一下如何让一个文件被git进行版本管理

1. 查看仓库的状态

```
git status
```

通过这个命令可以查看仓库相关状态，但是注意：这个仓库必须是 `git init` 之后的仓库，普通的文件夹是不能用 `git` 命令查看仓库状态的

```
ubuntu@xing:~/VGD_navigation/git/git_learn$ git status
fatal: 不是 git 仓库（或者任何父目录）：.git
```

```
ubuntu@xing:~/VGD_navigation/git/git_learn/my_repo$ git status
位于分支 master
无文件要提交，干净的工作区
```

如何判断是否是git仓库：通过 `ls -a` 来查看当前目录下是否有 `.git` 这个隐藏目录

2. 添加到暂存区

```
git add
```

- 可以使用通配符，例如： `git add *.txt`
- 也可以使用目录，例如： `git add .`

```
ubuntu@xing:~/VGD_navigation/git/git_learn/my_repo$ git status
位于分支 master
```

尚无提交

未跟踪的文件：

（使用 "`git add <文件>...`" 以包含要提交的内容）

```
file1.txt
file2.txt
file3.txt
file4.txt
file5.txt
```

提交为空，但是存在尚未跟踪的文件（使用 "`git add`" 建立跟踪）

这里可以看到这些文件在终端输出的是未跟踪的文件，说明还没有提交到暂存区，使用： `git add` 建立跟踪

```
ubuntu@xing:~/VGD_navigation/git/git_learn/my_repo$ git add .
ubuntu@xing:~/VGD_navigation/git/git_learn/my_repo$ git status
位于分支 master
```

尚无提交

要提交的变更:

(使用 "git rm --cached <文件>..." 以取消暂存)

```
新文件:      file1.txt
新文件:      file2.txt
新文件:      file3.txt
新文件:      file4.txt
新文件:      file5.txt
```

进行第一次提交之后再用 `git status` 查看相关文件状态，可以看到已经上传到暂存区了

3. 提交

只提交到暂存区中的内容，不会提交工作区相关内容

```
git commit -m <message of commit>
```

将暂存区相关的文件提交到本地仓库下，git就可以对其进行版本管理

```
ubuntu@xing:~/VGD_navigation/git/git_learn/my_repo$ git commit -m 第一次提交
[master (根提交) 494242d] 第一次提交
5 files changed, 5 insertions(+)
create mode 100644 file1.txt
create mode 100644 file2.txt
create mode 100644 file3.txt
create mode 100644 file4.txt
create mode 100644 file5.txt
```

提交到本地仓库之后就相当于git管理相关文件的版本了

4. 查看仓库提交历史记录

```
git log
```

可以查看仓库提交历史记录

```
ubuntu@xing:~/VGD_navigation/git/git_learn/my_repo$ git log
commit 494242d950b3f388f78367780b1df624d354ae4e (HEAD -> master)
Author: xing <2237200336@qq.com>
Date: Sat Oct 3 15:29:25 2026 +0800
```

第一次提交

同时 `--online` 参数来查看简洁的提交记录

```
ubuntu@xing:~/VGD_navigation/git/git_learn/my_repo$ git log --online
494242d (HEAD -> master) 第一次提交
```

五. git reset回退版本

```
git reset --soft <版本ID>
git reset --hard <版本ID>
git reset --mixed <版本ID>
#如果不加任何参数后缀, 就是默认--mixed
```

```
git ls-files
#这个命令是显示git仓库中受版本控制（已追踪）的文件列表
#默认: 列出暂存区和工作区中所有被git追踪的文件
```

git reset的三种模式

git reset --soft

git reset --hard

git reset --mixed

工作区



暂存区



1. git reset --soft

这个回退版本会保持相关工作区和暂存区文件的仍然存在, 只是将head指针回退到之前版本, 后续想恢复直接 `git`

```
commit -m <message of commit>
```

```
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo$ git log --oneline
c5c1b73 (HEAD -> master) 第三次提交
4f00163 第二次提交
7e4253d 第一次提交
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo$ git reset --soft 4f00163
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo$ git log --oneline
4f00163 (HEAD -> master) 第二次提交
7e4253d 第一次提交
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo$ git ls-files
file1.txt
file2.txt
file3.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo$ ls
file1.txt  file2.txt  file3.txt
```

```
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo$ git status
位于分支 master
要提交的变更:
  (使用 "git restore --staged <文件>..." 以取消暂存)
    新文件:   file3.txt
```

2. git reset --hard

慎用，这个会把暂存区和工作区相关文件全部删除不进行保存

```
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_hard$ git reset --hard 4f00163
HEAD 现在位于 4f00163 第二次提交
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_hard$ git ls-files
file1.txt
file2.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_hard$ ls
file1.txt  file2.txt
```

3. git reset --mixed

这种回退会保存工作区相关文件，但是暂存区不会保存相关文件

```
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_mixed$ git log --one-line
c5c1b73 (HEAD -> master) 第三次提交
4f00163 第二次提交
7e4253d 第一次提交
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_mixed$ git reset --mixed 4f00163
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_mixed$ git ls-files
file1.txt
file2.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_mixed$ ls
file1.txt file2.txt file3.txt
```

4. 回溯相关文件

利用 `git reflog` 查看之前提交的历史记录

再利用 `git reset --hard/soft/mixed <版本ID>` 进行回溯

六. git diff查看版本差异

1. 比较工作区和暂存区

```
git diff
```



```

ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ ls
file1.txt  file2.txt  file3.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git diff
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ nano file1.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git diff
diff --git a/file1.txt b/file1.txt
index 58c9bdf..ac3aed8 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1, +1 @@
-111
+this is VGD
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git add file1.t
xt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git status
位于分支 master
要提交的变更:
  (使用 "git restore --staged <文件>..." 以取消暂存)
    修改:      file1.txt

```

2.比较工作区+暂存区和本地仓库的差异

```
git diff HEAD
```

```

ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git add file3.t
xt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git diff HEAD
diff --git a/file3.txt b/file3.txt
index 55bd0ac..b1181e5 100644
--- a/file3.txt
+++ b/file3.txt
@@ -1, +1 @@
-333
+welcome VGD
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git commit -m "
修改2"
[master 94b947d] 修改2
 1 file changed, 1 insertion(+), 1 deletion(-)
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git diff HEAD

```

3. 比较暂存区和本地仓库的差异

```
git diff --cached/--staged
```

```
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ nano file2.tx
t
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git diff --ca
ched
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git add file2
.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git diff --ca
ched
diff --git a/file2.txt b/file2.txt
index c200906..e1d60d2 100644
--- a/file2.txt
+++ b/file2.txt
@@ -1, +1 @@
-222
+join our VGD
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git commit -m
error: switch `m' 需要一个值
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git commit
[master 5e18e03] 修改3
 1 file changed, 1 insertion(+), 1 deletion(-)
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git diff --ca
ched
```

4. 比较提交之间的差异

```
git diff <commit_ID> <commit_ID>
#一般是之前的版本ID和之后的版本ID进行比较
```

下面图片大家可以比较一下区别

```
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git diff 94b947d 5e18e03
diff --git a/file2.txt b/file2.txt
index c200906..e1d60d2 100644
--- a/file2.txt
+++ b/file2.txt
@@ -1,1 @@
-222
+join our VGD
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git diff 5e18e03 94b947d
diff --git a/file2.txt b/file2.txt
index e1d60d2..c200906 100644
--- a/file2.txt
+++ b/file2.txt
@@ -1,1 @@
-join our VGD
+222
```

```
git diff HEAD~n HEAD
```

这里讲解一下HEAD指针，HEAD指针一般指向你commit的最新节点的地址

- HEAD~n: n代表的是在HEAD指针前n个节点地址

```
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git diff HEAD
~3 HEAD
diff --git a/file1.txt b/file1.txt
index 58c9bdf..ac3aed8 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1,1 +1,1 @@
-111
+this is VGD
diff --git a/file2.txt b/file2.txt
index c200906..e1d60d2 100644
--- a/file2.txt
+++ b/file2.txt
@@ -1,1 +1,1 @@
-222
+join our VGD
diff --git a/file3.txt b/file3.txt
new file mode 100644
index 0000000..b1181e5
--- /dev/null
+++ b/file3.txt
@@ -0,0 +1,1 @@
```

七. 使用git rm删除文件

删除文件有两种方式：

1. 先删除工作区的文件再去上传到暂存区和本地仓库

```
rm files;
git add file;
git commit -m ...
```

```
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ ls
file1.txt  file2.txt  file3.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ rm file1.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ ls
file2.txt  file3.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git ls-files
file1.txt
file2.txt
file3.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git add .
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git ls-files
file2.txt
file3.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git commit -m
"删除"
[master a2b332e] 删除
1 file changed, 1 deletion(-)
delete mode 100644 file1.txt
```

2. 直接利用git命令

```
git rm <file>
#把文件从工作区和暂存区同时删除
git rm --cached <file>
#把文件从暂存区删除，但是保留在当前工作区中
git rm -r*
#递归删除某个目录下的所有子目录和文件，删除后不要忘记提交
```

对于能使用git命令的文件，其实相关的文件状态已经变成了git追踪的状态——即在暂存区或者本地仓库

```
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git reset --hard 5e18e03
HEAD 现在位于 5e18e03 修改3
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ ls
file1.txt  file2.txt  file3.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git ls-files
file1.txt
file2.txt
file3.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git rm file2.txt
rm 'file2.txt'
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ ls
file1.txt  file3.txt
ubuntu@xing:~/VGD_navigation/git/git_learn/new_repo_soft$ git ls-files
file1.txt
file3.txt
```

八. gitignore忽略文件

我们通常用 `.gitignore` 文件来忽略相关文件或者文件目录让其不进行版本管理

应该忽略哪些文件

.gitignore

- 忽略日志文件和文件夹 ✓
- 忽略所有.class文件 ✓
- 忽略所有.o文件 ✓
- 忽略所有.env文件 ✓
- 忽略所有.zip和tar文件 ✓
- 忽略所有.pem文件 ✓
-

- 系统或者软件自动生成的文件
- 编译产生的中间文件和结果文件
- 运行时生成日志文件、缓存文件、临时文件
- 涉及身份、密码、口令、秘钥等敏感信息文件

注意：

- .gitignore文件不会去忽略已经进行版本控制的相关文件或者文件目录
- 同时如果是空文件目录，git不会进行追踪和版本管理的

九. 关联本地仓库和远程仓库

- 推送到远程仓库：

```
git push <remote> <branch>
```

- 拉取远程仓库更新内容

```
git pull <remote>
#git pull会拉取远程仓库的更新并自动合并
git fetch <remote>
#git fetch会拉取但是不会自动合并到本地仓库中
```

远程仓库不进行过多讲解，在网上有很多相关的资料

1. 添加远程仓库：

```
#step1
git remote add <远程仓库别名> <远程仓库地址>
#step2
git push -u <远程仓库别名> <分支名>
```

2. 查看远程仓库：

```
git remote -v
```

3. 拉取远程仓库内容：

```
git pull <远程仓库别名> <远程仓库分支>:<本地分支名>(相同可省略)
```

注意：git上远程仓库别名和github/gittee上远程仓库名不同——一个是别名，一个是github/gittee上创建仓库的仓库名称

十. git分支操作

git分支操作：主要用于多人协作和管理项目对应的版本

1. 查看当前分支

```
git branch
```

2. 创建分支

```
git branch <branch_name>
```

3. 切换分支

```
git checkout <branch_name>
```

#注意: **checkout**命令会出现歧义, **checkout**不仅是切换分支, 还能恢复文件, 如果分支名字和文件名相同, 会导致相关歧义

```
git switch <branch_name>
```

#更推荐的方式进行切换分支

4. 合并分支

```
git merge <branch_name>
```

#将<branch_name>分支合并到当前工作的分支中

5. 删除分支

#删除已合并的分支

```
git branch -d <branch_name>
```

#删除未合并的分支

```
git branch -D <branch_name>
```

最后, git提供了看提交记录的图形化方式, 虽然没那么好看(

```
git log --oneline --graph --decorate --all
```

#可以用alias简化

```
alias graph="git log --oneline --graph --decorate --all"
```

十一. git分支合并冲突

两个分支未修改同一个文件的同一处位置: git自动合并

两个分支修改了同一个文件同一处位置, 产生冲突

发生合并冲突时, 可以尝试以下相关方式:

- 手动修改大法:
 - step1: 手动修改冲突文件, 手动合并冲突内容

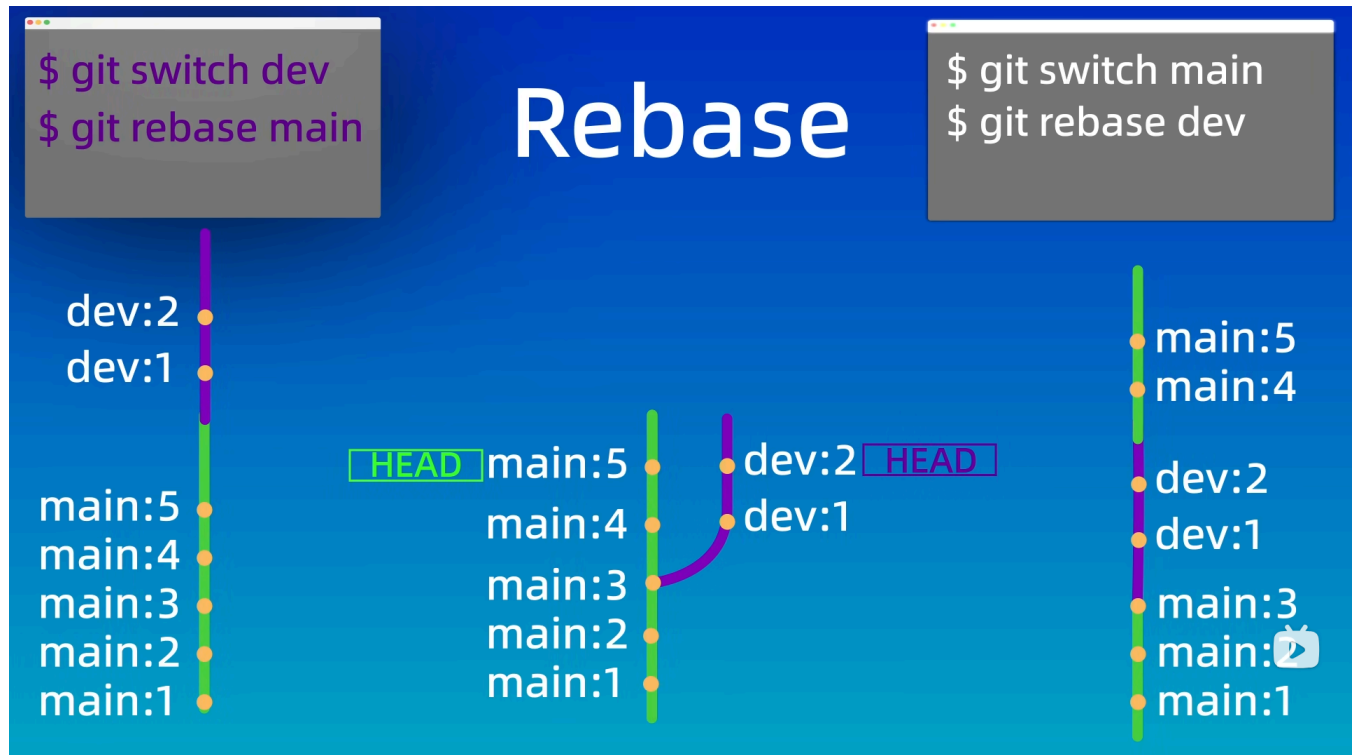
- step2: 添加到暂存区和提交修改
- 中止合并: 当不想继续执行合并操作时就使用以下命令来终止

```
git merge --abort
```

十二. git rebase

顾名思义: 就是变基

它会找到两个分支的共同节点, 然后将要变基的分支放到基的HEAD指针后面



很好的一张图片, 就是这个意思

```
git switch dev
git rebase main
```

切换到dev分支上然后找到dev和main分支上共同节点, 然后将dev分支变基到main的HEAD指针上

你要是问笔者merge和rebase对于分支操作的优缺, 嗯.....笔者不太清除, 因为本人没做过大型项目的多人开发(, 等后续慢慢体会吧

最后的最后: 附上一个**算法组**应该遵循的工作流——**github flow**

GitHub Flow

